

`ros2_impedance_controller`

v1.2.0

Leonardo Felipe dos Santos

September 2025

Contents

1.1	Introduction	2
1.2	Pinocchio URDF parsing	2
1.3	Classical impedance control law	2
1.3.1	Robot model	2
1.3.2	Cartesian impedance model	3
1.3.3	Simplifications	3

1.1 Introduction

The `ros2_impedance_controller` is a *ros2_control* controller that leverages ROS 2 interfaces and the Pinocchio rigid-body dynamics library. Its main goal is to provide a ROS 2-ready impedance controller that is robot-agnostic and can be easily integrated into ROS-based projects, whether in simulation (e.g., Gazebo) or on articulated robots such as manipulators or legged robots. For those unfamiliar with impedance control, here are some remarks:

- The actuator dynamics is neglected here. The robot must be able to apply the desired torque vector $\boldsymbol{\tau}_c$;
- The Jacobian is derived from the forward kinematic functions, $\boldsymbol{x} = f(\boldsymbol{q})$, and depends only of the joint configuration. No structural elastic effects are considered;
- The Cartesian impedance control is intrinsically a nonlinear computed-torque feedback law in the joint space, even for a linear impedance in the task space.

1.2 Pinocchio URDF parsing

TBD

1.3 Classical impedance control law

The control law implemented was optimized through mathematical simplifications. The law follows strictly the formalism on [1].

1.3.1 Robot model

Pinocchio algorithms adopts the Euler-Lagrange equation:

$$\boldsymbol{M}(\boldsymbol{q}) \ddot{\boldsymbol{q}} + \boldsymbol{C}(\boldsymbol{q}, \dot{\boldsymbol{q}}) \dot{\boldsymbol{q}} + \boldsymbol{g}(\boldsymbol{q}) = \boldsymbol{\tau} \quad (1.1)$$

where $\boldsymbol{q} \in \mathbb{R}^n$ is the generalized configuration vector for n degrees of freedom, $\boldsymbol{M}(\boldsymbol{q}) \succ 0, \in \mathbb{R}^{n \times n}$ is the generalized mass, $\boldsymbol{C}(\boldsymbol{q}, \dot{\boldsymbol{q}}) \in \mathbb{R}^{n \times n}$ is the Coriolis/centrifugal matrix, $\boldsymbol{g}(\boldsymbol{q}) \in \mathbb{R}^n$ is the vector of gravity forces, and $\boldsymbol{\tau} \in \mathbb{R}^n$ is the vector of generalized torques acting on the system.

1.3.2 Cartesian impedance model

At the interaction port (end-effector), the impedance model is the *mass-spring-damper* in Cartesian coordinates:

$$\mathbf{f}_{int} = \Lambda_d \ddot{\mathbf{e}} + \mathbf{D}_d \dot{\mathbf{e}} + \mathbf{K}_d \mathbf{e} \quad (1.2)$$

$$\mathbf{e} = \mathbf{x} - \mathbf{x}_d \quad (1.3)$$

Render (1.2), decoupled from the robot dynamics (1.1), is achieved by setting the joint torques as

$$\begin{aligned} \boldsymbol{\tau}_c = \mathbf{g}(\mathbf{q}) + \mathbf{J}(\mathbf{q})^T [\Lambda(\mathbf{x}) \ddot{\mathbf{x}}_d + \Gamma(\mathbf{x}, \dot{\mathbf{x}}) \dot{\mathbf{x}} - \\ \Lambda(\mathbf{x}) \Lambda_d^{-1} (\mathbf{D}_d \dot{\mathbf{e}} + \mathbf{K}_d \mathbf{e}) + (\Lambda(\mathbf{x}) \Lambda_d^{-1} - \mathbf{I}_k) \mathbf{f}_{int}] \end{aligned} \quad (1.4)$$

where $\Lambda(\mathbf{x})$ and $\Gamma(\mathbf{x}, \dot{\mathbf{x}})$ are the robot's generalized mass and Coriolis matrices in Cartesian coordinates, respectively, and $\mathbf{J}(\mathbf{q}) \in \mathbb{R}^{k \times n}$ is the geometric Jacobian. To avoid inertia shaping and force-feedback, one can use

$$\boldsymbol{\tau}_c = \mathbf{g}(\mathbf{q}) + \mathbf{J}(\mathbf{q})^T [\Lambda(\mathbf{x}) \ddot{\mathbf{x}}_d + \Gamma(\mathbf{x}, \dot{\mathbf{x}}) \dot{\mathbf{x}} - \mathbf{D}_d \dot{\mathbf{e}} - \mathbf{K}_d \mathbf{e}] \quad (1.5)$$

Equation (1.4) corresponds to the classical impedance control law, while (1.5) represents the variant without inertia shaping¹.

1.3.3 Simplifications

Given

$$\Lambda(\mathbf{x}) = \mathbf{J}(\mathbf{q})^{-T} \mathbf{M}(\mathbf{q}) \mathbf{J}(\mathbf{q})^{-1} \quad (1.6)$$

$$\Gamma(\mathbf{x}, \dot{\mathbf{x}}) = \mathbf{J}(\mathbf{q})^{-T} \left(\mathbf{C}(\mathbf{q}, \dot{\mathbf{q}}) - \mathbf{M}(\mathbf{q}) \mathbf{J}(\mathbf{q})^{-1} \dot{\mathbf{J}}(\mathbf{q}) \right) \mathbf{J}(\mathbf{q})^{-1} \quad (1.7)$$

(1.4) simplifies to:

$$\begin{aligned} \boldsymbol{\tau}_c = \mathbf{g} + \mathbf{M} \mathbf{J}^{-1} \ddot{\mathbf{x}}_d + (\mathbf{C} - \mathbf{M} \mathbf{J}^{-1} \dot{\mathbf{J}}) \mathbf{J}^{-1} \dot{\mathbf{x}} - \\ \mathbf{M} \mathbf{J}^{-1} \Lambda_d^{-1} (\mathbf{D}_d \dot{\mathbf{e}} + \mathbf{K}_d \mathbf{e}) + (\mathbf{M} \mathbf{J}^{-1} \Lambda_d^{-1} - \mathbf{J}) \mathbf{f}_{int} \end{aligned}$$

¹It is important to note that it is more challenging to achieve stability guarantees when dealing with a varying inertia.

$$\tau_c = g + M J^{-1} \ddot{x}_d + (C - M J^{-1} \dot{J}) \dot{q} + M J^{-1} \Lambda_d^{-1} (f_{int} - D_d \dot{e} - K_d e) - J f_{int}, \quad (1.8)$$

and (1.5) becomes:

$$\tau_c = g(q) + M J^{-1} \ddot{x}_d + (C - M J^{-1} \dot{J}) \dot{q} - J(q)^T (D_d \dot{e} + K_d e) \quad (1.9)$$

These computationally improved versions of (1.4) and (1.5) are in the `update()` method:

```

298 jsim_jpinv_ = robot_data_>M * jacobian_pinv_;
299 jsim_jpinv_dj_ = jsim_jpinv_ * jacobian_derivative_;
300
301 actual_inertia_ = jacobianT_pinv_ * jsim_jpinv_;
302
303 impedance_wrench_.noalias() =
304     (desired_stiffness_ * pose_deviation_ + desired_damping_ * ↵
305         twist_deviation_) * (-1);
306
307 if (inertia_shaping_)
308 {
309     impedance_torques_.noalias() =
310         jsim_jpinv_ * desired_inertia_inv_ * (impedance_wrench_ + ↵
311             sensor_wrench_);
312     impedance_torques_.noalias() -= jacobian_.transpose() * sensor_wrench_ ↵
313         ;
314 }
315 else
316 {
317     impedance_torques_.noalias() = jacobian_.transpose() * ↵
318         impedance_wrench_;
319     desired_inertia_ = actual_inertia_;
320 }
321
322 accel_feedforward_ = jsim_jpinv_ * desired_pose_accel_;
323 twist_compensation_.noalias() = (robot_data_>C - jsim_jpinv_dj_) * ↵
324     robot_velocities_;
325 effort_commands_ = accel_feedforward_ + impedance_torques_ + ↵
326     twist_compensation_ + robot_data_>g;

```

Bibliography

- [1] C. Ott, *Cartesian Impedance Control of Redundant and Flexible-Joint Robots*, ser. Springer Tracts in Advanced Robotics. Springer, 2008.